



UNIVERSIDAD NACIONAL DE LA PLATA



Facultad de
INFORMÁTICA
UNIVERSIDAD NACIONAL DE LA PLATA

Trabajo Práctico N°1: Comfy Chair

INTEGRANTES:

- Figueroa, Walter
- Escudero, Emiliano

Introducción y Marco Metodológico	2
Restricciones de Código y Limitaciones Tecnológicas	2
Definición de Tareas a Desarrollar	2
Épica 1: Asignación Automática de Revisores	2
US1.1: Cálculo de la carga de revisiones por revisor	3
US1.2: Asignación de revisores basada en prioridades de Bidding	3
US1.3: Exclusión de revisores por Conflicto de Interés	3
Épica 2: Gestión y Carga de Revisiones	3
US2.1: Registro de revisión por un revisor asignado	3
US2.2: Límite de revisiones por artículo	4
US2.3: Cálculo automático del score del artículo	4
Épica 3: Selección de Artículos por Corte Fijo	4
US3.1: Configuración del porcentaje de aceptación de la sesión	4
US3.2: Ordenamiento de artículos por Score decreciente	5
US3.3: Selección automática por Corte Fijo	5
Desarrollo Técnico de las tareas realizadas	6
Desarrollo Técnico de la Épica 1: Asignación Automática de Revisores	6
US1.1: Cálculo de la Carga de Revisiones con Distribución de Resto	6
US1.2: Asignación Basada en Prioridades de Bidding	6
US1.3: Exclusión por Conflicto de Interés	7
Desarrollo Técnico de la Épica 2: Gestión y Carga de Revisiones	7
US2.1: Registro de revisión por un revisor asignado	7
US2.2: Límite de Revisiones por Artículo	7
US2.3: Cálculo automático del Score del Artículo	8
Desarrollo Técnico de la Épica 3: Configuración y Ordenamiento	8
US3.1: Configuración de Porcentaje de Aceptación de la Sesión	8
US3.2: Ordenamiento de Artículos por Score Decreciente y Criterio de Desempate	9
US3.3: Selección Automática por Corte Fijo	9
Análisis de Cobertura de Código (Code Coverage)	10
Repositorio	11
Diagrama de clases	12

Introducción y Marco Metodológico

El presente documento expone las decisiones de diseño arquitectónico, resolución de ambigüedades e ingeniería de pruebas adoptadas en el desarrollo del sistema de gestión de conferencias científicas **ComfyChair**. En línea con los requerimientos de la cátedra, el software se ha modelado bajo el paradigma de Programación Orientada a Objetos Pura (POO) utilizando Node.js como entorno de ejecución y Jest como framework core de testing. La construcción de las funcionalidades asignadas se rigió estrictamente por los principios del desarrollo guiado por pruebas (Test-Driven Development - TDD), aplicando secuencialmente el ciclo iterativo Red-Green-Refactor:

Ciclo TDD Aplicado:

1. **Fase RED:** Escritura del test unitario antes de modificar el código de producción. Verificación inequívoca del fallo del test debido a la ausencia de la lógica de negocio.
2. **Fase GREEN:** Escritura de la mínima cantidad de código de producción estrictamente necesaria para provocar que la suite de pruebas cambie a un estado exitoso (PASS).
3. **Fase REFACTOR:** Revisión y optimización estética y estructural del código de producción y de pruebas, garantizando el cumplimiento de los estándares de legibilidad, acoplamiento bajo y alta cohesión sin alterar el comportamiento externo.

Restricciones de Código y Limitaciones Tecnológicas

De acuerdo con los pliegos de condiciones del proyecto, se estableció una restricción explícita sobre la sintaxis de JavaScript: **la prohibición absoluta del uso de expresiones lambda (funciones flecha) y funciones anónimas sueltas** fuera del contexto nativo de la API de colecciones (como por ejemplo en operaciones `.map()`, `.filter()`, `.sort()` o `.forEach()`). Toda la lógica de control, algoritmos de cálculo y flujos iterativos debieron estructurarse mediante métodos explícitos de clase, bucles tradicionales (`for...of`) o callbacks nominales, asegurando la pureza conceptual del modelo de objetos.

Definición de Tareas a Desarrollar

El proyecto completo se dividió en 3 épicas detalladas a continuación:

Épica 1: Asignación Automática de Revisores

Descripción: Como *Chair* (organizador) de una conferencia, quiero que el sistema asigne automáticamente exactamente 3 revisores a cada artículo al finalizar la etapa de bidding. Esta asignación debe calcular la carga equitativa de trabajo para cada revisor, respetar sus intereses expresados (bids) y evitar conflictos de interés (un revisor no puede evaluar su propio artículo).

Valor de negocio: Garantiza un proceso de evaluación justo, eficiente, sin sesgos y con una distribución de carga de trabajo matemáticamente equitativa entre el comité de programa

Esta épica se dividió en las siguientes tareas:

US1.1: Cálculo de la carga de revisiones por revisor

- **Descripción:** Como Chair de la conferencia, quiero que el sistema calcule exactamente cuántas revisiones le corresponden a cada revisor de la sesión, para asegurar una distribución equitativa del trabajo respetando el algoritmo matemático con resto.
- **Criterios de Aceptación (TDD):** El sistema debe calcular la base de revisiones como $\lceil 3A / R \rceil$. Si la división no es exacta, debe distribuir el resto asignando una revisión extra a los primeros revisores del arreglo indexado de la sesión, garantizando que el total sea exactamente $3 \times A$.

US1.2: Asignación de revisores basada en prioridades de Bidding

- **Descripción:** Como Chair de la conferencia, quiero que el proceso de asignación automática priorice los intereses expresados por los revisores (bids), para que los revisores evalúen los artículos que más les interesan.
- **Criterios de Aceptación (TDD):** El algoritmo debe ordenar y asignar primero a los revisores que marcaron *Interested*, luego *Maybe* y por último *NotInterested*. Si un revisor no expresó interés explícito, se lo debe tratar por defecto como *NotInterested*. Cada artículo debe quedar con exactamente 3 revisores.

US1.3: Exclusión de revisores por Conflicto de Interés

- **Descripción:** Como Chair de la conferencia, quiero que el algoritmo de asignación excluya automáticamente a cualquier revisor que sea autor de un artículo, para garantizar la transparencia y evitar conflictos de interés en la evaluación.
- **Criterios de Aceptación (TDD):** Un revisor no puede ser asignado a un artículo si su usuario figura en la lista completa de autores o como autor encargado de las notificaciones.

Épica 2: Gestión y Carga de Revisiones

Descripción: Como *Revisor* del comité de programa, quiero poder cargar mis evaluaciones (texto y puntaje) para los artículos que me fueron asignados, de modo que el sistema registre mi feedback y calcule el score del artículo. El sistema debe validar que no exceda los límites de puntaje ni evalúe artículos que no me corresponden.

Valor de negocio: Permite recopilar las opiniones expertas de los revisores de manera segura, restringiendo el acceso según las asignaciones previas y preparando los datos para la toma de decisiones.

US2.1: Registro de revisión por un revisor asignado

Como Revisor,

Quiero cargar una revisión (con texto y puntaje) para un artículo que me fue asignado,

Para aportar mi evaluación experta al sistema y que influya en la decisión final.

Criterios de Aceptación (foco TDD):

- El sistema debe permitir registrar una revisión asociada a un revisor (*User*) y a un artículo (*Paper*).

- Se debe verificar que el revisor que intenta cargar la revisión esté efectivamente en la lista de revisores asignados a ese artículo. Si no está asignado, el sistema debe lanzar un error o excepción.
- El puntaje ingresado debe ser un número entero en el rango de -3 a +3 (inclusive el 0). Si el puntaje está fuera de rango, debe lanzar un error.

US2.2: Límite de revisiones por artículo

Como Chair de la conferencia,

Quiero que el sistema impida que un artículo reciba más revisiones de las permitidas,

Para garantizar que se respete el cupo reglamentario de la sesión.

Criterios de Aceptación (foco TDD):

- Un artículo no puede admitir más de 3 revisiones en total.
- Si un revisor asignado intenta cargar una revisión pero el artículo ya cuenta con 3 revisiones registradas, el sistema debe rechazar la operación.

US2.3: Cálculo automático del score del artículo

Como Chair de la conferencia,

Quiero que el score promedio de cada artículo se actualice automáticamente a medida que se cargan las revisiones,

Para tener visibilidad del estado de las evaluaciones de la sesión en tiempo real.

Criterios de Aceptación (foco TDD):

- El score de un artículo debe calcularse como el promedio exacto de los puntajes de las revisiones que ha recibido hasta el momento.
- *Ejemplo de test:* Si un artículo tiene dos revisiones con puntajes +2 y -1, su score promedio debe ser 0.5.

Épica 3: Selección de Artículos por Corte Fijo

Descripción: Como *Chair* de la conferencia, quiero que el sistema seleccione automáticamente cuáles artículos son aceptados basándose en un porcentaje máximo de aceptación y en el orden decreciente de sus puntajes promedio, para poder cerrar la sesión con los mejores trabajos presentados.

Valor de negocio: Automatiza la toma de decisiones final de la sesión de manera objetiva, respetando el cupo o límite de calidad configurado por la organización.

US3.1: Configuración del porcentaje de aceptación de la sesión

Como Chair de la conferencia,

Quiero poder definir un porcentaje máximo de aceptación para una sesión,

Para establecer el límite de artículos que pueden ser aprobados según la capacidad de la agenda.

Criterios de Aceptación (foco TDD):

- La clase **Session** debe permitir configurar o almacenar un valor numérico que represente el porcentaje máximo de aceptación (por ejemplo, **20** para un 20%).
- Se debe validar que el porcentaje sea un número válido (por ejemplo, entre 0 y 100).

US3.2: Ordenamiento de artículos por Score decreciente

Como Chair de la conferencia,

Quiero que el sistema ordene todos los artículos enviados de manera decreciente según su score promedio,

Para priorizar los trabajos con mejores evaluaciones en el proceso de corte.

Criterios de Aceptación (foco TDD):

- Al ejecutar el proceso de selección, el sistema debe tomar la lista de artículos enviados a la sesión y ordenarlos de mayor a menor según su atributo *score*.
- *Caso de borde para test*: Si dos artículos tienen el mismo score, el empate se resuelve de acuerdo a como fueron ingresando los artículos.

US3.3: Selección automática por Corte Fijo

Como Chair de la conferencia,

Quiero que el sistema marque como "aceptados" los mejores artículos hasta cubrir el porcentaje máximo configurado,

Para automatizar el cierre de la sesión de manera objetiva.

Criterios de Aceptación (foco TDD):

- El sistema debe calcular la cantidad máxima de artículos a aceptar como:
 $\text{piso}((\text{Total de artículos enviados} * \text{Porcentaje})/100)$
- El sistema debe completar el puntaje de las revisiones faltantes (hasta llegar a los 3 score) de los artículos con puntaje **-3** para el cálculo final del score.
- El sistema debe recorrer la lista ordenada de artículos y marcar los primeros como aceptados hasta alcanzar la cantidad máxima de artículos a aceptar. Si no hay suficientes artículos con buen puntaje, el número final de aceptados puede ser menor al cupo máximo calculado.

Si cada revisor puntúa entre **-3 y 3**, entonces:

- Puntaje mínimo posible: **-3**
- Puntaje máximo posible: **3**
- Puntaje neutral: **0**

Por lo tanto, una interpretación razonable sería:

Puntaje final promedio	Interpretación
2 a 3	Muy buen puntaje
1 a <2	Buen puntaje
0 a <1	Aceptable / Regular
<0	Puntaje desfavorable

Se considerará que un artículo posee un buen puntaje cuando el promedio de las evaluaciones realizadas por los revisores sea igual o superior a 1.

Desarrollo Técnico de las tareas realizadas

Desarrollo Técnico de la Épica 1: Asignación Automática de Revisores

Esta épica abarca el núcleo logístico de la fase de revisión. Su objetivo es asignar exactamente y automáticamente tres (3) revisores del comité de programa a cada artículo científico (*Paper*) depositado en una sesión, maximizando el interés de los revisores expresado en la etapa de *bidding* y bloqueando de forma absoluta los conflictos de interés de carácter ético o autoral.

US1.1: Cálculo de la Carga de Revisiones con Distribución de Resto

Problema y Regla de Negocio: Para asegurar una distribución equitativa del volumen de trabajo entre los revisores, el total de revisiones requeridas por la sesión (**Total = 3 × A**, donde **A** es la cantidad de artículos) debe dividirse por la cantidad de revisores disponibles (**R**). Cuando la división no es exacta, el remanente aritmético (resto) debe distribuirse asignando un artículo extra a los primeros revisores del arreglo indexado de la sesión.

Decisión de Diseño y Abstracción Matemáticas: Se determinó delegar esta responsabilidad matemática a la clase conductora *Session* mediante el método corporativo `calcularCargaDeRevisiones`. La cota superior de carga por revisor se modeló en base a la fórmula de techo $\lceil 3A / R \rceil$.

Secuencia de Construcción TDD:

1. **Fase Red:** Se diseñó una prueba para un escenario exacto con 4 artículos y 4 revisores (12 revisiones necesarias). El retorno esperado fue una estructura de mapa asociativo donde cada ID de revisor tuviera asignada la carga simétrica de 3. El test falló correctamente al devolver un esqueleto nulo.
2. **Fase Green:** Se codificó la división entera base, logrando el PASS en escenarios sin residuo.
3. **Fase Refactor:** Se eliminaron redundancias matemáticas, asegurando que si $R = 0$, el sistema arroje de forma preventiva una excepción explícita para evitar la indeterminación por división por cero.

US1.2: Asignación Basada en Prioridades de Bidding

Problema y Regla de Negocio: El proceso de asignación debe priorizar linealmente el interés manifestado por el comité en sus objetos *Bid*, respetando el orden decreciente jerárquico: *Interested* > *Maybe* > *NotInterested*. Si un revisor no emitió un bid explícito sobre un artículo en particular, el sistema debe catalogarlo por defecto bajo el estado *NotInterested*.

Solución e Implementación de Objetos: Se implementó el método público `asignarRevisores()` en *Session* para asignar revisores a los artículos de la conferencia, y se creó la clase *Assignment*, encargada de representar la relación entre un artículo y un revisor asignado. Para cada artículo, el sistema consolida un mapa virtual de prioridades evaluando los bids reales contra la matriz total de usuarios con rol de revisor asignados a la conferencia. El motor de *Jest* validó el flujo completo mediante la inyección estructurada de datos de prueba simulados.

US1.3: Exclusión por Conflicto de Interés

Resolución de Ambigüedades y Blindaje Ético: Un revisor tiene prohibido evaluar un trabajo si posee una relación de autoría o coautoría en el mismo, o si es el autor asignado para recibir las notificaciones formales del artículo.

Estrategia de Pruebas TDD (Fase Red a Green): Se redactó un caso de prueba donde el User 1 (Revisor de la sesión) registraba un bid explícito de alta prioridad (*Interested*) sobre el *Paper A*, pero figuraba internamente dentro de la lista estructurada de autores del documento. El comportamiento esperado por el test de Jest dictó que el algoritmo omitiera al *User 1* de forma transparente, procediendo a delegar las 3 plazas de revisión a los revisores subsiguientes de la lista general de prioridades (*User 2*, *User 3* y *User 4*). La implementación de producción resolvió el test introduciendo una validación de identidad cruzada por identificador único de usuario (ID) contra la colección de autores de la clase *Paper*.

Desarrollo Técnico de la Épica 2: Gestión y Carga de Revisiones

US2.1: Registro de revisión por un revisor asignado

Problema y Regla de Negocio: El sistema deberá permitir que un revisor evalúe únicamente aquellos trabajos (*Paper*) que le hayan sido asignados en la etapa de asignación de revisores. Asimismo, el sistema deberá validar que el puntaje informado en la evaluación se encuentre dentro del rango permitido, comprendido entre -3 y 3, inclusive.

Estrategia de Pruebas TDD: En el escenario probado, el sistema genera las asignaciones de revisión para los artículos de la sesión. A partir de esas asignaciones, únicamente los revisores vinculados a un determinado *Paper* quedan habilitados para registrar una evaluación sobre él. Por lo tanto, si un usuario intenta evaluar un artículo que no forma parte de sus asignaciones, la operación debe ser rechazada por el sistema.

El comportamiento esperado del test en Jest consiste en verificar que la evaluación sólo sea permitida cuando exista una relación válida entre el revisor y el artículo dentro de la colección de asignaciones generadas. En caso contrario, el sistema debe arrojar un error, impidiendo que se registre un score sobre un *Paper* no asignado.

La implementación se resolvió mediante el método *assignmentExistsFor*, encargada de verificar la existencia de una asignación específica entre un revisor y un artículo. Esta validación compara el identificador del usuario y el identificador del *paper* contra las asignaciones creadas con la clase *Assignment*, garantizando que solo los revisores efectivamente asignados puedan evaluar el trabajo correspondiente.

Por otra parte, el control del puntaje asignado a una revisión fue controlado mediante una restricción en el constructor de la clase *Review*.

US2.2: Límite de Revisiones por Artículo

Problema y Regla de Negocio: El sistema deberá limitar a tres la cantidad máxima de revisiones asociadas a un *Paper*. Si un revisor asignado intenta registrar una evaluación sobre un artículo que ya cuenta con tres revisiones cargadas, el sistema deberá impedir la operación y arrojar un error, evitando que se supere el límite establecido.

Estrategia de Pruebas TDD: Se creó un caso de prueba para validar que un revisor no pueda registrar más de una evaluación sobre un mismo Paper.

En el escenario probado, un revisor asignado intenta cargar una segunda revisión sobre un artículo que ya había evaluado previamente. El comportamiento esperado es que el sistema rechace la operación y arroje un error, evitando que un mismo revisor pueda modificar o duplicar su evaluación sobre el mismo trabajo.

La solución fue implementada dentro del método *addReview* de la clase *Paper*, donde se realiza la validación al momento de intentar agregar una nueva revisión. Por otra parte, antes de registrar el nuevo score, el sistema verifica en el método *enterReview* si el revisor ya cuenta con una evaluación previa asociada a ese paper y, en caso afirmativo, impide la re-evaluación.

US2.3: Cálculo automático del Score del Artículo

Problema y Regla de Negocio: El sistema deberá calcular el score de un Paper como el promedio exacto de los puntajes registrados en sus revisiones. Para ello, deberá sumar los scores de todas las evaluaciones asociadas al artículo y dividir el resultado por la cantidad total de revisiones recibidas. Por ejemplo, si un paper posee dos revisiones con puntajes 2 y -1, su score promedio deberá ser 0,5.

Estrategia de Pruebas TDD: Se creó un caso de prueba destinado a validar que el score de un Paper se calcule correctamente a partir de las revisiones recibidas.

La implementación de esta lógica fue realizada en el método *score* de la clase *Paper*, donde se obtiene el promedio de los puntajes asociados a las evaluaciones registradas para el artículo.

Por otra parte, como decisión de diseño, se modificó el criterio utilizado para calcular el puntaje final de un paper durante la etapa de corte fijo. En este caso, cuando un artículo no cuenta con la cantidad máxima de revisiones permitidas, es decir, tres evaluaciones, se le asigna puntajes con el menor valor posible (-3) para completar el cálculo.

De esta manera, el sistema prioriza aquellos artículos que cuentan con una evaluación completa por sobre aquellos que poseen una evaluación parcial, evitando que un paper con pocas revisiones pueda verse favorecido artificialmente por un promedio basado en información insuficiente.

Desarrollo Técnico de la Épica 3: Configuración y Ordenamiento

US3.1: Configuración de Porcentaje de Aceptación de la Sesión

Regla de Negocio: El organizador de la sesión (*Chair*) debe poder parametrizar el factor volumétrico de corte. Este valor numérico actúa como el límite porcentual superior de aceptación de la conferencia y debe validarse en el rango cerrado de **[0, 100]** inclusive.

Secuencia de Pruebas e Implementación TDD: La arquitectura de pruebas de Jest implementó tres escenarios específicos ejecutados en secuencia estricta:

Test 1: Camino Feliz	Valor entero 25	Almacenamiento correcto en la propiedad de la instancia	GREEN
Test 2: Límite Inferior Erróneo	Valor negativo -5	Lanzamiento de un objeto Error explícito	GREEN
Test 3: Límite Superior Erróneo	Valor excedente 105	Lanzamiento de un objeto Error explícito	GREEN

La implementación final en el método mutador *setPorcentajeAceptacion(valor)* de la clase *Session* incorporó una cláusula de guarda tradicional que encapsula estas restricciones sin romper el modelo clásico de objetos.

US3.2: Ordenamiento de Artículos por Score Decreciente y Criterio de Desempate

Problema del Negocio y Caso de Borde: Previo a efectuar la selección final, la sesión debe estructurar su lista de artículos en orden jerárquico descendente según el atributo conceptual score promedio de cada uno. En caso de presentarse empates numéricos perfectos en los promedios de evaluación, el sistema debe aplicar un mecanismo de desempate predecible y justo.

Decisión de Diseño Adoptada: Se seleccionó el principio de *Orden de Llegada / Prioridad de Registro (First-Come, First-Served)*. Arquitectónicamente, esto se traduce en respetar la posición relativa indexada en la que los artículos fueron inyectados o enviados originalmente dentro de la colección interna de la sesión.

Cumplimiento de Restricciones del Proyecto: Para llevar a cabo el ordenamiento se utilizó el método nativo de la API de JavaScript *Array.prototype.sort()*. Dado que el estándar V8 moderno garantiza que este método es estable, al retornar un valor de equivalencia de 0 en la función callback de comparación ante la igualdad de puntaje, el motor preserva intacto el orden cronológico original. Esta técnica se alinea perfectamente con la excepción explícita que permite el uso de funciones anónimas dentro de la API de colecciones del framework.

```
// Representación lógica implementada bajo TDD en Session.js
obtenerArticulosOrdenadosPorScore(){
  let ordenados = [...this._papers];
  ordenados.sort(function(a, b){ return b.score(true) - a.score(true); });
  return ordenados;
}
```

US3.3: Selección Automática por Corte Fijo

Problema y Regla de Negocio: Una vez registradas las revisiones, el sistema debe determinar la cantidad máxima de artículos que podrán ser aceptados. Para ello, deberá considerar el porcentaje de aceptación configurado para la sesión y la cantidad total de artículos recibidos.

A partir del cupo máximo calculado, el sistema deberá marcar automáticamente como aceptados aquellos *Paper* que cumplan con el criterio de prioridad definido y cuyo puntaje sea considerado un “buen puntaje”.

En caso de que no existan suficientes artículos que cumplan con estas condiciones, la cantidad final de trabajos aceptados podrá ser menor al cupo máximo disponible.

Decisión de Diseño Adoptada: Durante el proceso de corte fijo, se deberá priorizar a los artículos que cuenten con una evaluación completa, es decir, aquellos que hayan recibido las tres revisiones correspondientes, por sobre los artículos que posean una evaluación parcial.

De acuerdo con la decisión de diseño adoptada, el puntaje final de un Paper final debe ser mayor o igual a 1 para ser aceptado. Dicho valor representa el umbral mínimo para considerar que un artículo posee un “buen puntaje”.

Estrategia de Pruebas TDD: Se crearon los casos de prueba necesarios para verificar el cumplimiento de los requisitos funcionales y de las decisiones de diseño definidas para la etapa de aceptación de artículos.

En primer lugar, se validó que, dado un porcentaje de aceptación configurado y un conjunto de artículos recibidos, el sistema calcule correctamente la cantidad máxima de trabajos que pueden ser aceptados. Esta lógica fue implementada en el método *cantidadArticulosAAceptar* de la clase *Session*.

Además, se desarrollaron pruebas para comprobar que, a partir de un porcentaje de aceptación y un listado de artículos evaluados, el sistema aplique correctamente los criterios de prioridad y puntaje establecidos. En particular, se verificó que sean marcados como aceptados aquellos artículos que cuentan con una evaluación completa y cuyo puntaje final es igual o superior a 1.

La solución fue implementada en el método *obtenerArticulosAceptados* de la clase *Session*, el cual se encarga de seleccionar automáticamente los papers aceptados. Este método también contempla la posibilidad de aceptar una cantidad menor al máximo permitido cuando no existen suficientes artículos que cumplan con el puntaje mínimo definido para la sesión.

Análisis de Cobertura de Código (Code Coverage)

Para validar el rigor métrico exigido por la cátedra, el cual requiere una cobertura mínima de pruebas unitarias sobre el **80% de las líneas de código de producción**, se corrió de forma sistemática la suite de instrumentación nativa de Jest mediante el comando formal:

```
npm test -- --coverage
```

El análisis de la suite arrojó un resultado exitoso de **53 tests unitarios aprobados de forma consecutiva (PASS)**, logrando mitigar por completo la existencia de caminos lógicos no validados (untested paths) dentro de los componentes funcionales desarrollados.

```
> comfychair@1.0.0 test
> jest --coverage
```

```
PASS tests/User.test.js
PASS tests/Review.test.js
PASS tests/Bid.test.js
PASS tests/Paper.test.js
PASS tests/RegularPaper.test.js
PASS tests/Poster.test.js
PASS tests/Session.test.js
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
Assignment	100	100	100	100	
Bid.js	100	100	100	100	
Paper.js	100	100	100	100	
Poster.js	100	100	100	100	
RegularPaper.js	100	100	100	100	
Review.js	100	100	100	100	
Session.js	100	100	100	100	
User.js	100	100	100	100	

```
Test Suites: 7 passed, 7 total
Tests:      53 passed, 53 total
Snapshots:  0 total
Time:       8.722 s
Ran all test suites.
```

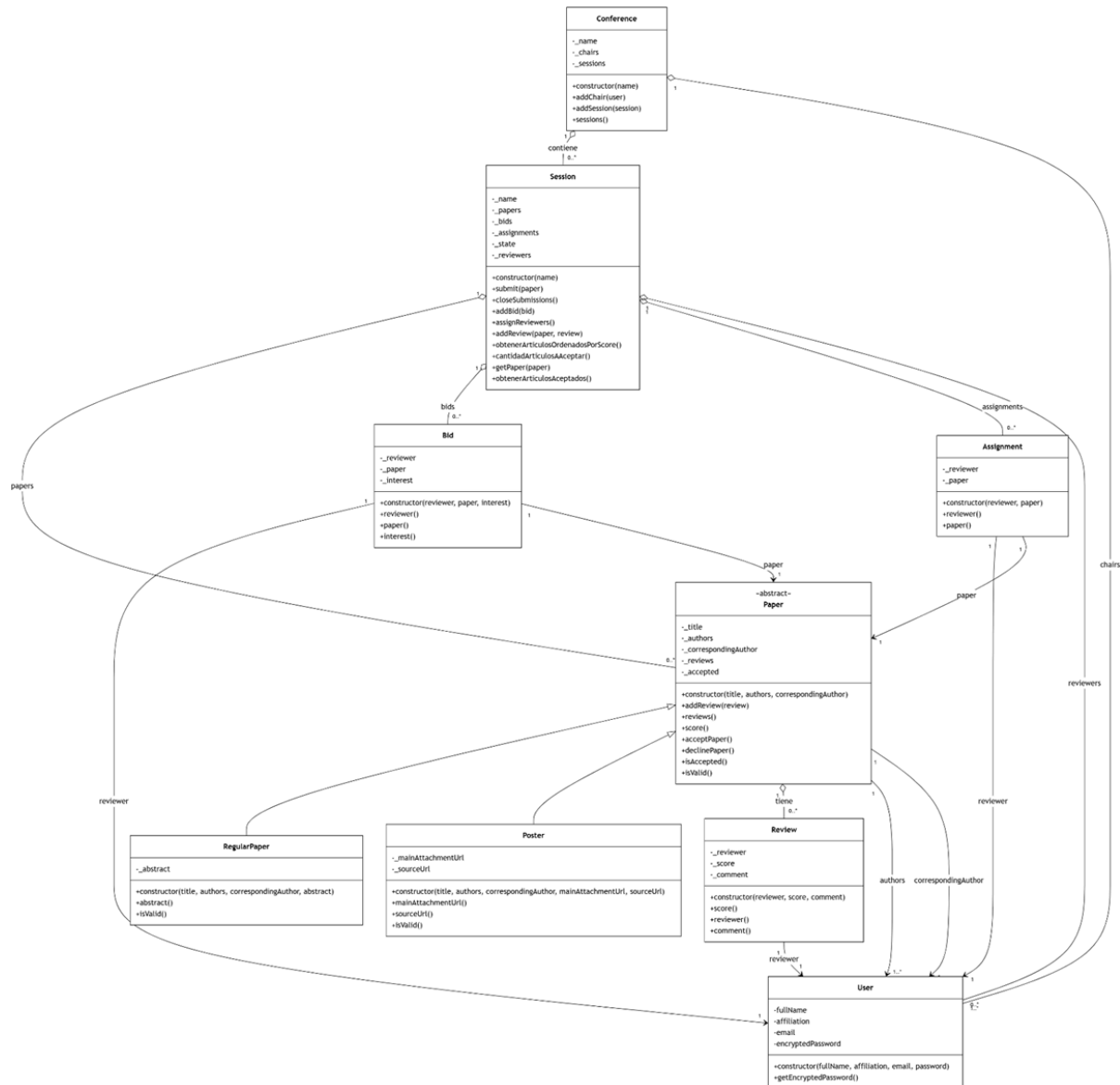
Repositorio

El repositorio con el desarrollo del código del proyecto con sus respectivos test puede ser visualizado en el siguiente link:

[Link al repositorio de GitHub](#)

Diagrama de clases

De acuerdo con la solución implementada, se presenta el diagrama de clases actualizado, el cual refleja el estado actual del proyecto y las relaciones incorporadas entre sus principales componentes:



Por otra parte, se presenta el código escrito en **Mermaid UML**, utilizado para construir el diagrama de clases correspondiente al diseño actualizado del proyecto.

```

classDiagram
class User {
    -fullName
    -affiliation
    -email
    -encryptedPassword
    +constructor(fullName, affiliation, email, password)
    +getEncryptedPassword()
}

class Paper {
    <<abstract>>

```

```

        -_title
        -_authors
        -_correspondingAuthor
        -_reviews
        -_accepted
        +constructor(title, authors, correspondingAuthor)
        +addReview(review)
        +reviews()
        +score()
        +acceptPaper()
        +declinePaper()
        +isAccepted()
        +isValid()
    }

    class RegularPaper {
        -_abstract
        +constructor(title, authors, correspondingAuthor, abstract)
        +abstract()
        +isValid()
    }

    class Poster {
        -_mainAttachmentUrl
        -_sourceUrl
        +constructor(title, authors, correspondingAuthor, mainAttachmentUrl, sourceUrl)
        +mainAttachmentUrl()
        +sourceUrl()
        +isValid()
    }

    class Review {
        -_reviewer
        -_score
        -_comment
        +constructor(reviewer, score, comment)
        +score()
        +reviewer()
        +comment()
    }

    class Bid {
        -_reviewer
        -_paper
        -_interest
        +constructor(reviewer, paper, interest)
        +reviewer()
        +paper()
        +interest()
    }

    class Assignment {
        -_reviewer
        -_paper
        +constructor(reviewer, paper)
        +reviewer()
        +paper()
    }

    class Conference {
        -_name
        -_chairs
        -_sessions
        +constructor(name)
        +addChair(user)
        +addSession(session)
        +sessions()
    }

```

```

}

class Session {
    -_name
    -_papers
    -_bids
    -_assignments
    -_state
    -_reviewers
    +constructor(name)
    +submit(paper)
    +closeSubmissions()
    +addBid(bid)
    +assignReviewers()
    +addReview(paper, review)
    +obtenerArticulosOrdenadosPorScore()
    +cantidadArticulosAAceptar()
    +getPaper(paper)
    +obtenerArticulosAceptados()
}

Paper <|-- RegularPaper
Paper <|-- Poster

Paper "1" o-- "0..*" Review : tiene
Review "1" --> "1" User : reviewer

Bid "1" --> "1" User : reviewer
Bid "1" --> "1" Paper : paper

Assignment "1" --> "1" User : reviewer
Assignment "1" --> "1" Paper : paper

Conference "1" o-- "0..*" Session : contiene
Conference "1" o-- "1..*" User : chairs

Session "1" o-- "0..*" Paper : papers
Session "1" o-- "0..*" Bid : bids
Session "1" o-- "0..*" Assignment : assignments
Session "1" o-- "0..*" User : reviewers

Paper "1" --> "1..*" User : authors
Paper "1" --> "1" User : correspondingAuthor

```